This simple "infinite" loop will continue to prompt for user input until a specific string is detected from STDIN. Here's what it looks like running:

```
$ php while2.php
Enter 'stop' to terminate
hello
You entered hello

Enter 'stop' to terminate
stop
Terminating.
```

The expression of a `while` loop is checked before any potential execution of the statements in the `while` block. If you need to run the block at least once, then `do` can be placed at the start of the block and `while` at the end – commonly known as a `do-while` structure:

```
do {
    print "I'm doing a thing." . PHP_EOL;
} while (false);
print "I've done a thing.";

Output:
I'm doing a thing.
I've done a thing.
```

Nesting, `break`, `continue`

We've seen how we can use `break` to exit a loop. Don't forget that `break` also stops execution inside a `switch` block (conceptually, a `switch` statement is like a loop that has an iteration for each `case` statement). It is possible to have loops within loops – an example of such a construct can be found in Chapter 5, "Arrays," for processing multi-dimensional arrays. `break` statements can have numeric values appended to them to indicate how many levels of nested looping should be terminated:

```
$multiArr = [
    ['duck', 'duck'],
    ['duck', 'goose', 'duck'],
```